SQL ASSIGNMENT 13

Subject: Database Management Systems (SQL) | **Topic:** Value & Aggregating Window Functions (LAG, LEAD, Running Total & Moving Avg) | **Status:** Completed

QUESTION 1

Create a table named `Orders` (order_id, user_id, order_date, total_amount) and insert at least 7 sample rows representing different users and order dates (Zomato/Swiggy order history).

```
CREATE TABLE Orders ( order_id INT PRIMARY KEY, user_id INT, order_date DATE, total_amount DECIMAL(10, 2) ); INSERT INTO Orders (order_id, user_id, order_date, total_amount) VALUES (501, 201, '2026-10-01', 450.00), (502, 201, '2026-10-05', 620.00), (503, 201, '2026-10-12', 310.00), (504, 201, '2026-10-20', 890.00), (505, 202, '2026-10-02', 150.00), (506, 202, '2026-10-10', 280.00), (507, 202, '2026-10-18', 540.00);
```

QUESTION 2

Write a SQL query using the `LAG()` function to display each user's order_id, order_date, and the total_amount of their previous order.

```
SELECT order_id, user_id, order_date, total_amount, LAG(total_amount, 1) OVER ( PARTITION BY user_id ORDER BY order_date ) AS previous_order_amount FROM Orders;
```

QUESTION 3

Write a SQL query using the `LEAD()` function to display each user's order_id, order_date, and the total_amount of their next order.

```
SELECT order_id, user_id, order_date, total_amount, LEAD(total_amount, 1) OVER ( PARTITION BY user_id ORDER BY order_date ) AS next_order_amount FROM Orders;
```

QUESTION 4

Calculate the cumulative running total of `total_amount` per user across their order history using window framing.

```
SELECT order_id, user_id, order_date, total_amount, SUM(total_amount) OVER ( PARTITION BY user_id ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW ) AS running_total FROM Orders;
```

QUESTION 5

Calculate a 3-order moving average of `total_amount` for each user using `AVG() OVER()` framed across 2 preceding rows and the current row.

```
SELECT order_id, user_id, order_date, total_amount, AVG(total_amount) OVER ( PARTITION BY user_id ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW ) AS moving_avg FROM Orders;
```